

Ch.4 Kubernetes 시작하기

빔 프로젝터 화면에 통합 사이트가 명령 하나로 올라갔습니다. 프론트, 백엔드, DB 컨테이너가 줄줄이 올라오는 로그를 팀장이 팔짱을 낀 채 바라봤습니다. 마지막 빌드 줄이 떨어지고 브라우저에 회원 목록이 떴을 때, 팀장이 의자 등받이에서 몸을 떼며 한마디를 던졌습니다.

팀장 : "오픈 씨, 구성은 깔끔하네요. 그런데 이거 운영 서버에 올릴 거잖아요. 만약 새벽 두 시에 컨테이너 하나가 죽으면 누가 살리죠?"

질문이 끝나기도 전에 입이 막혔습니다. 자리에서 모니터를 들여다보고 있을 때라면 다시 띄우면 그만입니다. 자리를 비웠거나 잠들어 있을 때는 누가 알람을 받아도 한참 뒤에야 손을 댈 수 있습니다. 도커는 컨테이너를 띄워 주는 도구이지, 죽은 컨테이너를 감시하며 자동으로 복구하는 도구는 아닙니다.

팀장의 질문은 거기서 끝나지 않았습니다.

- 새벽에 컨테이너 한 대가 죽으면 누가 자동으로 살릴 수 있는지
- 사용자가 갑자기 몰리면 서버 대수를 어떻게 늘릴 건지
- 새 버전을 배포할 때 서비스 중단 없이 교체할 수 있는지

팀장 : "이제 운영까지 생각해야죠. 쿠버네티스(Kubernetes) 한번 제대로 파 봐요."

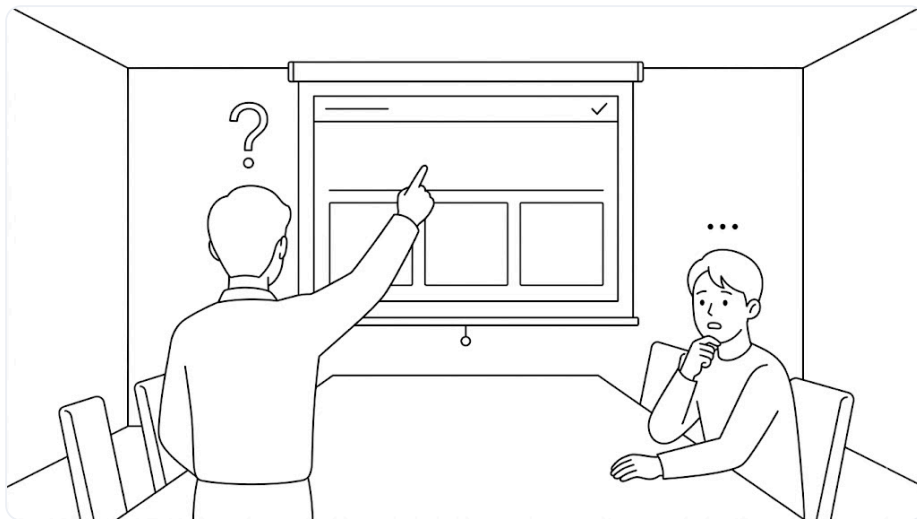


그림 4-1. 사이트는 떴지만, 팀장의 운영 질문에 답이 막힌 순간

회의가 끝나고 자리로 돌아왔지만 모니터 글자는 잘 들어오지 않았습니다. 그동안 도커로 한 일이 결국 **컨테이너 실행**까지였다는 게 그제야 보였습니다. 퇴근하고 집에서 노트북을 열었습니다. 도커에 겨우 익숙해졌는데 또 산을 넘어야 한다니 막막했지만, 팀장이 깔아 놓은 운영의 숙제를 풀려면 쿠버네티스가 왜 필요한지부터 짚어야 했습니다.

4.1 Kubernetes - Docker만으로 부족한 순간

4.1.1 도커는 실행 명령, K8s는 약속 유지

팀장의 질문은 사실 오픈이가 이미 다 겪어 본 일들이었습니다. 새벽 알람도, 이벤트 시작과 동시에 몰린 트래픽도, 배포 중 멈춘 결제 화면도 모두 운영 단계에서 발생한 문제입니다.

쿠버네티스를 알아보니 도커와 컨테이너를 다루는 방식 자체가 달랐습니다.

- **Docker:** "이 컨테이너를 지금 띄워라." 한 번의 실행 지시입니다.
- **Kubernetes:** "이 서비스 3개가 항상 떠 있게 유지해라." 지속되는 **약속**입니다.

쿠버네티스는 약속한 상태를 24시간 감시하다가 어긋나면 즉시 새 컨테이너를 실행시켜 다시 맞춥니다. 오픈이가 매번 직접 명령어를 쳐서 상태를 만들던 방식과는 다른 **관리** 방식이었습니다.

'내가 일일이 감시하고 살리지 않아도 시스템이 알아서 관리해 준다는 거네.'

4.1.2 본사가 원하는 상태를 선언한다

선언형이라는 단어는 책에서 자주 봤습니다. 그런데 시스템이 "계속 비교하며 상태를 맞춘다"는 게 막상 어떤 모습인지 그림이 그려지지 않았습니다. 다음 날 사무실에서 옆자리 선배에게 슬쩍 물어봤습니다.

선배 : "프랜차이즈 본사가 가맹점을 어떻게 관리하는지 생각하면 편해요. 본사가 매장에 들어가서 직접 요리하지는 않잖아요."

본사는 가맹점 하나하나를 직접 운영하지 않습니다. 대신 "서울 지역에 가맹점 50개 유지" 같은 지침을 내려보냅니다. 한 가맹점이 갑자기 폐업하면 관리팀이 새 가맹점을 여는 방향으로 움직이고, 손님이 몰려 대기 줄이 길어지면 직원을 더 뽑으라고 지시합니다. 본사는 '**원하는 상태(Desired State)**' 만 선언하고, 현장은 그 상태에 맞춰 끊임없이 움직입니다.

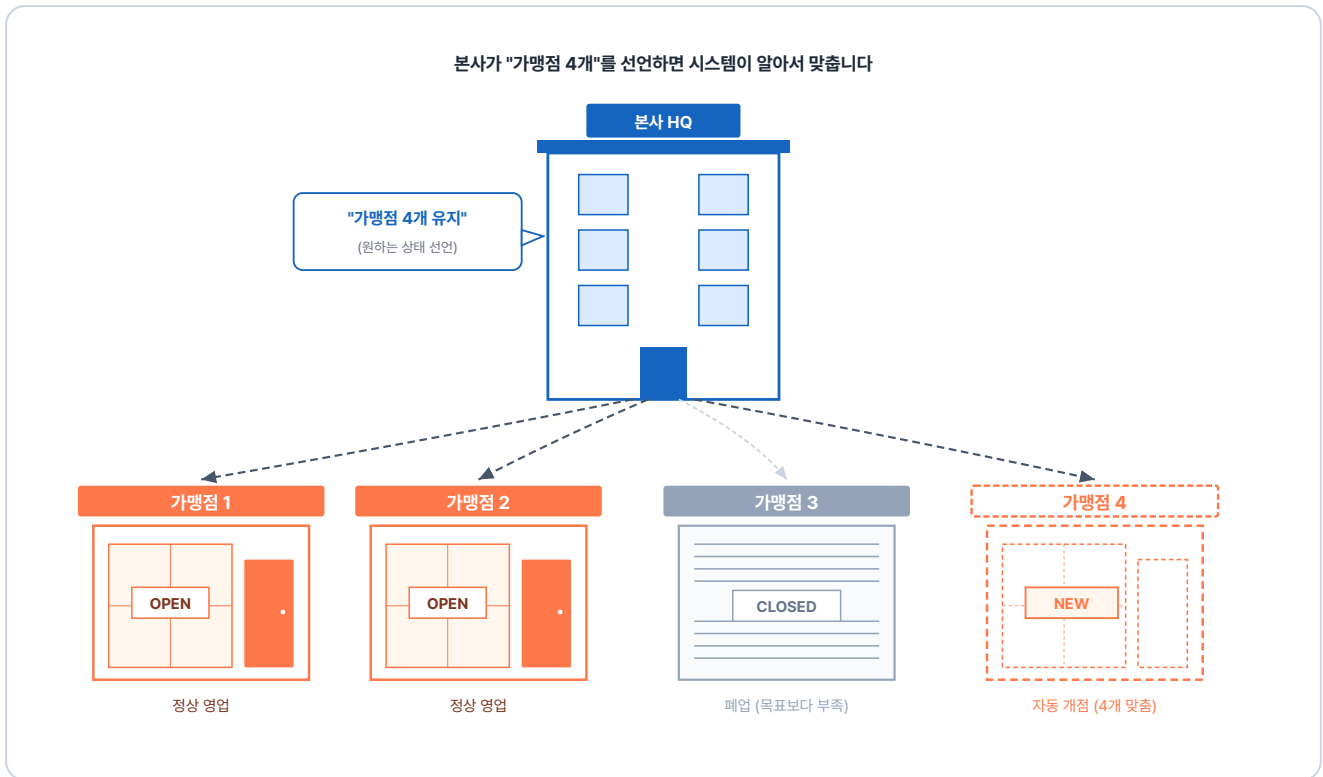


그림 4-2. 본사가 "가맹점 4개 유지"를 선언하면 시스템이 가맹점 개수를 자동으로 맞추는 구조

쿠버네티스도 마찬가지입니다. 오픈이가 "Pod 4개 유지"를 선언만 해 두면, 시스템이 그 상태를 자동으로 맞춥니다.

4.1.3 K8s 핵심 리소스 한눈에

쿠버네티스 세상에서는 모든 작업 단위를 **리소스(Resource)** 라고 부릅니다. 사용자의 요청은 여러 리소스를 거쳐 실제 컨테이너에 도달합니다. 전체적인 흐름은 다음과 같습니다.

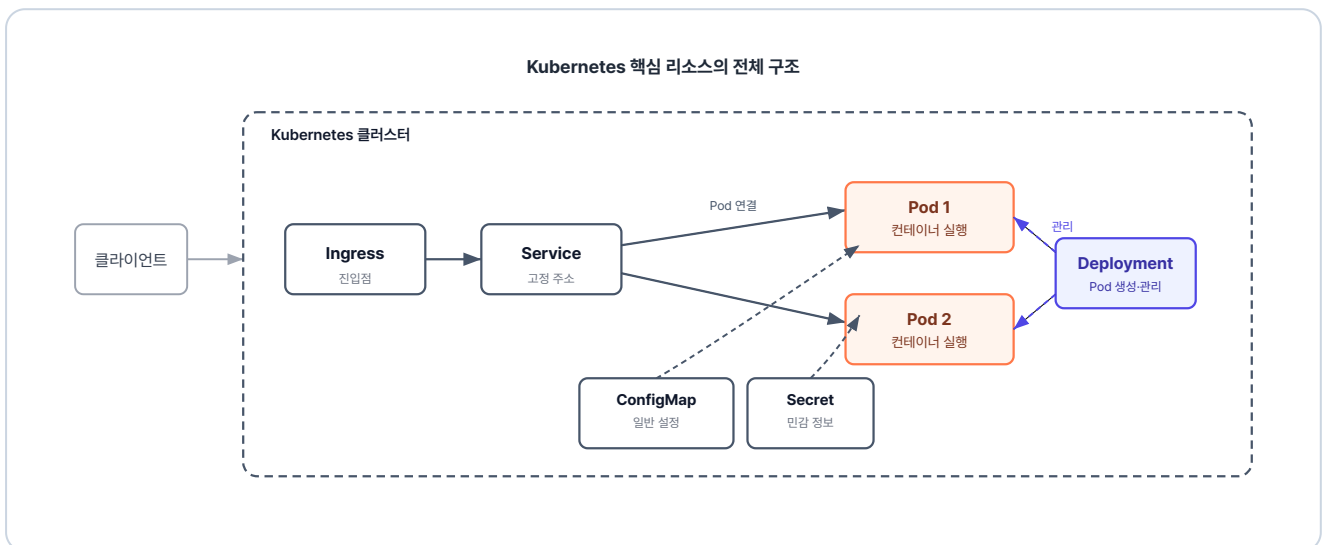


그림 4-3. Kubernetes 핵심 리소스의 전체 구조

이번 챕터에서는 가장 기본이 되는 Pod와 Deployment를 먼저 다뤄 보겠습니다. 자동 복구와 스케일링 같은 핵심 기능을 이 둘만으로도 충분히 구현할 수 있기 때문입니다.

4.1.4 K8s 동작 원리 - 컨트롤 플레인과 워커 노드의 조직도

쿠버네티스는 크게 **컨트롤 플레인(Control Plane)** 과 **워커 노드(Worker Node)** 라는 두 조직으로 나뉩니다. 이 둘이 합쳐져 유기적으로 돌아가는 전체 시스템을 **클러스터(Cluster)** 라고 부릅니다.

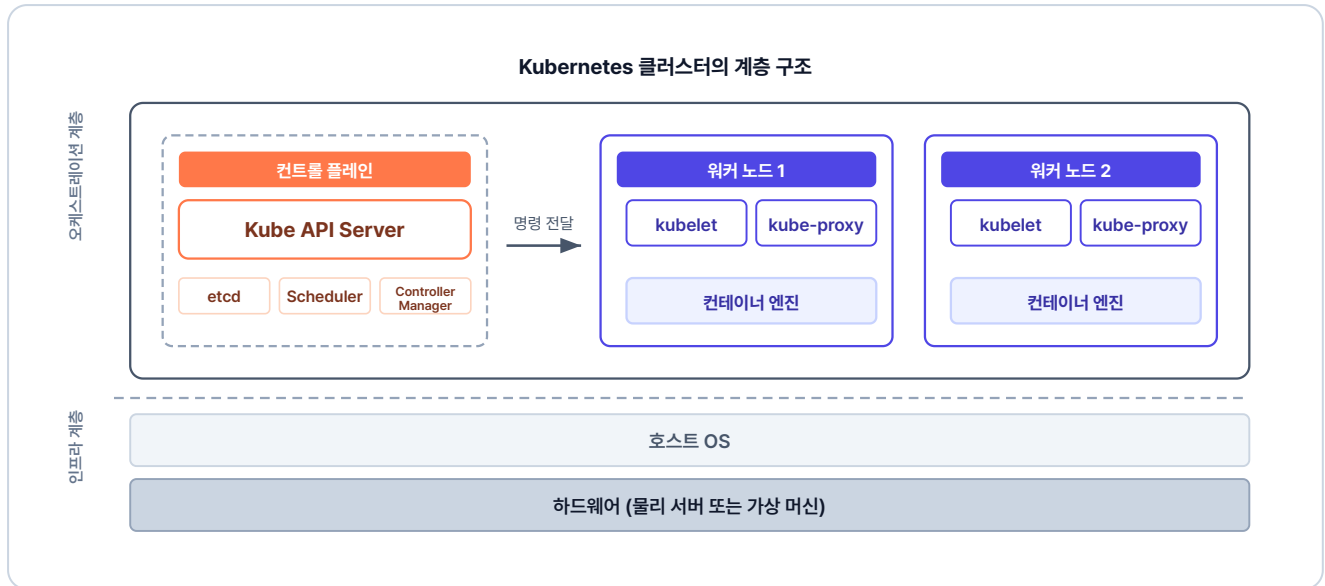


그림 4-4. Kubernetes 클러스터의 구조

- **클러스터:** 컨트롤 플레인과 모든 워커 노드를 하나로 묶은 전체 시스템입니다.
- **컨트롤 플레인:** 클러스터 전체를 관리하는 시스템입니다. 개발자의 명령을 받아 Pod를 어느 워커 노드에 띄울지 결정하고, 장애가 생긴 Pod를 새 Pod로 자동 교체하며, 클러스터 상태를 항상 원하는 모습으로 유지합니다.
- **워커 노드:** Pod가 실제로 띄워져서 일을 처리하는 노드입니다. 컨트롤 플레인의 결정에 따라 Pod를 띄우고 운영합니다.

노드(Node): 컨테이너가 실제로 돌아가는 컴퓨터 한 대를 말합니다. 실제 서비스 환경에서는 수십~수백 대의 노드(서버)가 모여 하나의 클러스터를 이룹니다.

개발자가 명령을 내리면 **본사 관리팀(컨트롤 플레인)**에서는 접수, 판단, 지시가 순차적으로 일어납니다.

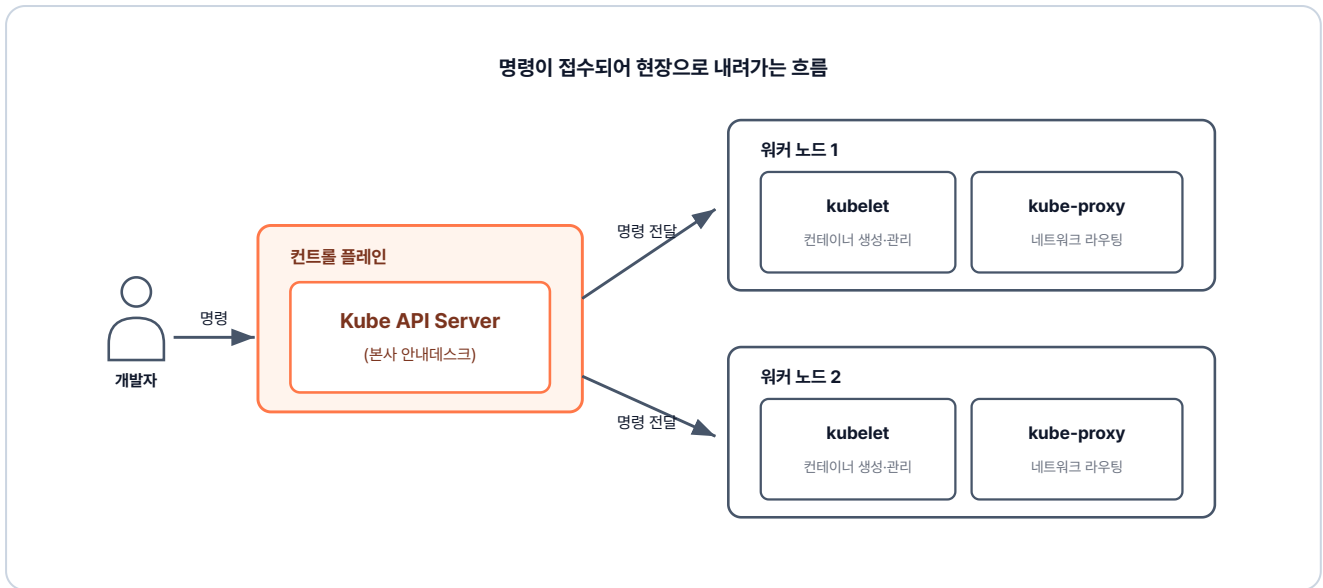


그림 4-5. 명령이 접수되어 현장으로 내려가는 흐름

1. **접수**: 개발자의 명령이 **Kube API Server(본사 안내데스크)** 로 들어옵니다.
2. **판단**: 컨트롤 플레인 내부에서 부서들이 바빠집니다. 새 Pod를 어느 워커 노드에 배치할지 **위치를 정하고 (Scheduler)** Pod가 정의대로 **잘 운영되는지 감시(Controller Manager)** 합니다.
3. **실행**: 확정된 지시가 해당 워커 노드의 **Kubelet**에게 전달되어 실제 운영에 반영됩니다.

이름이 조금 낯설어도 괜찮습니다. "**컨트롤 플레인**은 **계획(선언)**을 받고, 워커 노드는 그 계획대로 **Pod를 운영한다**" 는 흐름만 챙기면 됩니다. 컨트롤 플레인 안에서 어떤 부서들이 일을 나누는지 한 단계 더 들여다보겠습니다.

컨트롤 플레인 안에는 네 개의 핵심 부서가 있습니다.

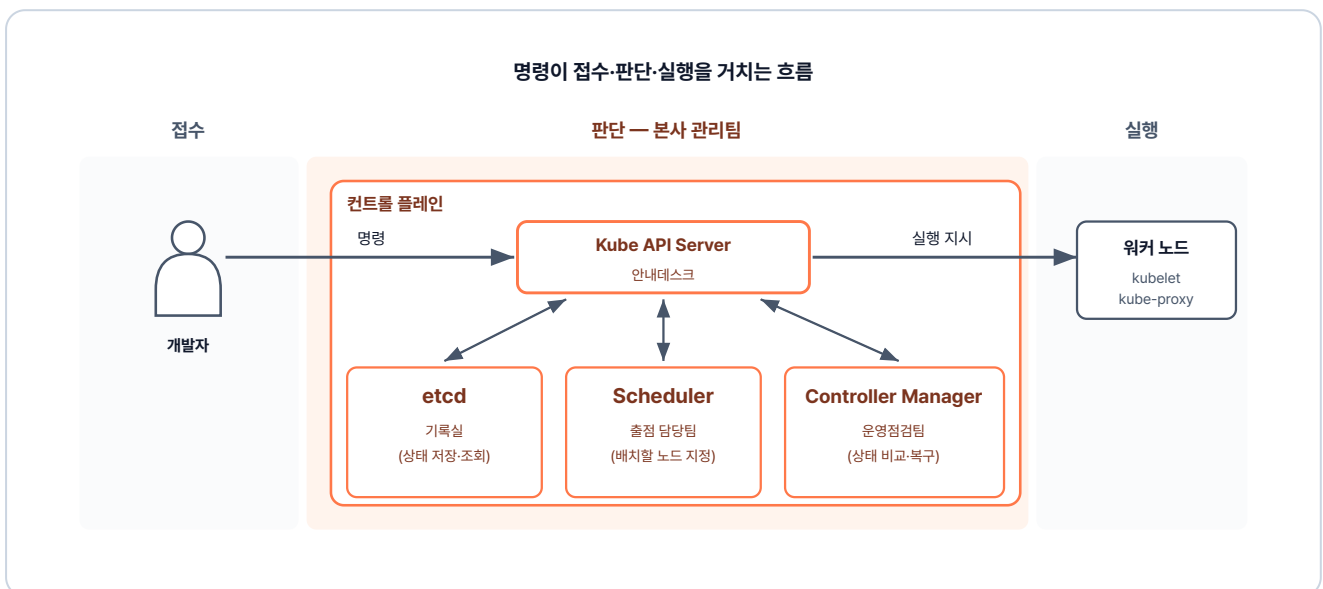


그림 4-6. 명령이 접수·판단·실행을 거치는 흐름

1. **Kube API Server(안내데스크)** 가 명령을 받으면 그 내용을 **etcd(기록실)** 에 저장합니다.
2. 기록된 정보를 바탕으로 **Scheduler(출점 담당팀)** 은 새 가맹점을 어느 노드에 둘지 정하고, **Controller Manager(운영점검팀)** 은 운영 중인 가맹점이 지침대로 돌아가는지 점검합니다.
3. 두 팀의 판단은 다시 Kube API Server를 거쳐 워커 노드로 내려갑니다.

구성 요소	비유	역할
Kube API Server	본사 안내데스크	모든 요청이 가장 먼저 도달하는 입구입니다. 외부 명령과 내부 부서 간의 소통이 모두 이 통로를 거칩니다.
etcd	본사 기록실	클러스터의 원하는 상태와 현재 상태를 모두 기록·조회하는 데이터베이스입니다.
Scheduler	출점 담당팀	새 Pod를 어느 워커 노드에 배치할지 결정합니다.
Controller Manager	운영점검팀	원하는 상태와 실제 상태를 끊임없이 비교하다가 차이가 생기면 복구 작업을 지시합니다.

워커 노드 쪽에는 컨트롤 플레인의 지시를 받아 컨테이너를 띄우는 **kubelet**과, 노드 안에서 들어오는 요청을 살아있는 Pod로 이어 주는 **kube-proxy**가 일합니다. 자세한 동작은 5장에서 다룹니다.

'본사가 큰 그림을 그리고, 현장에서 실무가 돌아간다... 구조는 알겠는데, 내 노트북 한 대로 이 거대한 시스템을 돌릴 수 있을까?'

4.2 Minikube - 로컬에 세우는 작은 클러스터

4.2.1 노트북 한 대로 클러스터 흉내 내기

컨트롤 플레인과 워커 노드 구조를 잡으려면 서버가 여러 대 필요할 것 같지만, 다행히 연습 단계에서는 노트북 한 대로도 충분합니다. Minikube를 쓰면 내 컴퓨터 안에 쿠버네티스 환경을 통째로 구축할 수 있습니다.

실제 쿠버네티스와 구조는 똑같지만 한 가지 차이가 있습니다. 노드가 단 하나라는 점입니다. Minikube는 노트북 안에 가상 머신(VM)이나 컨테이너를 하나 띄우고, 그 안에 컨트롤 플레인과 워커 노드 기능을 몽땅 집어넣습니다. 프랜차이즈로 치면 본점밖에 없는 구조입니다.

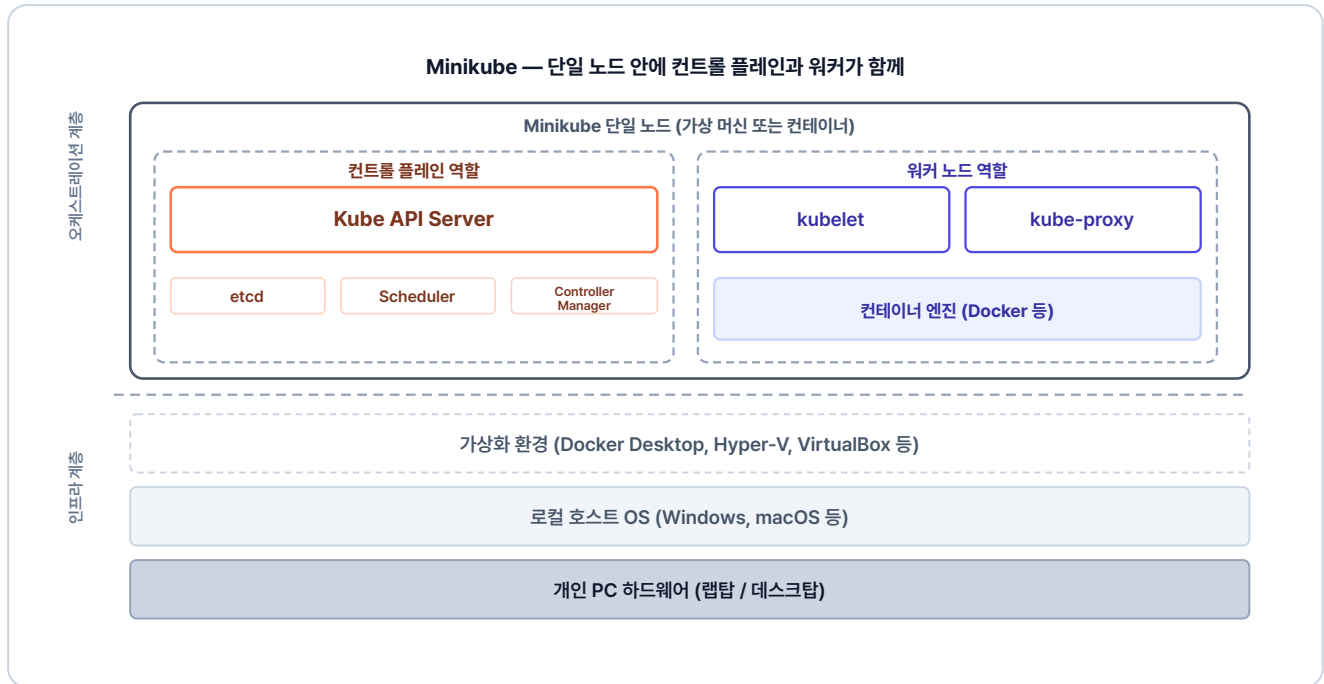


그림 4-7. Minikube는 단일 노드 안에 컨트롤 플레인과 워커 노드 기능이 함께 들어간 구조

노드가 하나라 구조가 가볍고 리소스도 적게 먹습니다. 클라우드 전용의 복잡한 기능은 제한적이지만, 이번에 배울 Pod나 Deployment 같은 핵심 리소스를 익히기에는 이만한 게 없습니다. 여기서 연습한 설정 파일은 나중에 실제 운영 서버로 옮겨도 거의 그대로 쓸 수 있습니다.

Minikube: Mini + Kubernetes의 합성어로, 개인 PC에서 쿠버네티스를 실습하기 위한 표준 도구입니다. 가벼운 가상 환경을 사용해 클러스터를 흉내 냅니다.

'서버 없이 노트북 한 대로 클러스터를 돌릴 수 있다니 다행이네. 이제 하나씩 시작해 봐야겠다.'

오픈이는 한 번 기지개를 켜 뒤 새 터미널 창을 열었습니다.

4.2.2 설치와 시작

각 운영체제(OS)에 맞는 패키지 관리자로 Minikube를 설치합니다. Windows는 **Chocolatey**, Mac은 **Homebrew** 패키지 관리자가 미리 설치돼 있어야 합니다.

```
# Windows (관리자 권한 터미널)
choco install minikube

# Mac
brew install minikube
```

명령어를 치고 잠시 기다리면 노트북 안에 쿠버네티스 환경이 구성됩니다. 이제 클러스터를 띄워 첫 명령을 내려 볼 차례입니다.

```
minikube start          # 클러스터 시작
```

실행 결과

```
$ minikube start
* Microsoft Windows 11 Pro 10.0.26200.6584 Build 26200.6584 의 minikube v1.37.0
* 자동적으로 docker 드라이버가 선택되었습니다. 다른 드라이버 목록: hyperv, ssh
* Docker Desktop 드라이버를 루트 권한으로 사용 중
* "minikube" 클러스터의 "minikube" primary control-plane 노드를 시작하는 중
* 기본 이미지 v0.0.48를 가져오는 중 ...
* docker container (CPUs=2, 메모리=3500MB) 를 생성하는 중 ...-
* 쿠버네티스 v1.34.0 을 Docker 28.4 런타임으로 설치하는 중###
* bridge CNI (Container Networking Interface) 를 구성하는 중 ...
* Kubernetes 구성 요소를 확인...
  - 이미지 gcr.io/k8s-minikube/storage-provisioner:v5 사용 중
* 애드온 활성화 : storage-provisioner, default-storageclass
* 끝났습니다! kubectl이 "minikube" 클러스터와 "default" 네임스페이스를 기본적으로 사용하도록 구성되었습니다
```

그림 4-8. minikube start 실행 결과

'쿠버네티스 테스트를 해 볼 준비가 끝났네. 생각보다 간단하구나.'

4.2.3 자주 쓰는 Minikube 명령어

오픈이는 실습 중 자주 쓰게 될 명령어들을 미리 정리해 뒀습니다.

명령어	설명
<code>minikube start</code>	클러스터를 시작합니다.
<code>minikube stop</code>	실행 중인 클러스터를 중지합니다.
<code>minikube status</code>	현재 클러스터의 구동 상태를 확인합니다.
<code>minikube service <서비스명> --url</code>	생성한 서비스에 접근할 수 있는 URL을 생성합니다.

4.3 첫 Pod - Kubernetes의 최소 단위

노트북에 Minikube 환경을 구성했으니 이제 실제 쿠버네티스의 리소스를 하나씩 띄워 볼 차례입니다. 도커만 쓸 때와 무엇이 다른지, 쿠버네티스라는 시스템이 컨테이너를 다루는 최소 단위부터 확인해 보겠습니다.

4.3.1 kubectl과 Pod

클러스터에 명령을 내릴 때 사용하는 도구는 kubectl입니다. "이런 상태를 만들어 달라"고 쿠버네티스에 요청하는 명령줄 도구입니다.

본격적인 실습 전에 오픈이는 **Pod(파드)** 라는 생소한 단위를 먼저 짚고 넘어갔습니다. 도커에서는 컨테이너가 실행의 최소 단위지만, 쿠버네티스는 컨테이너를 직접 제어하지 않습니다. 대신 하나 이상의 컨테이너를 Pod라는 상자로 한 번 감싸서 관리합니다.

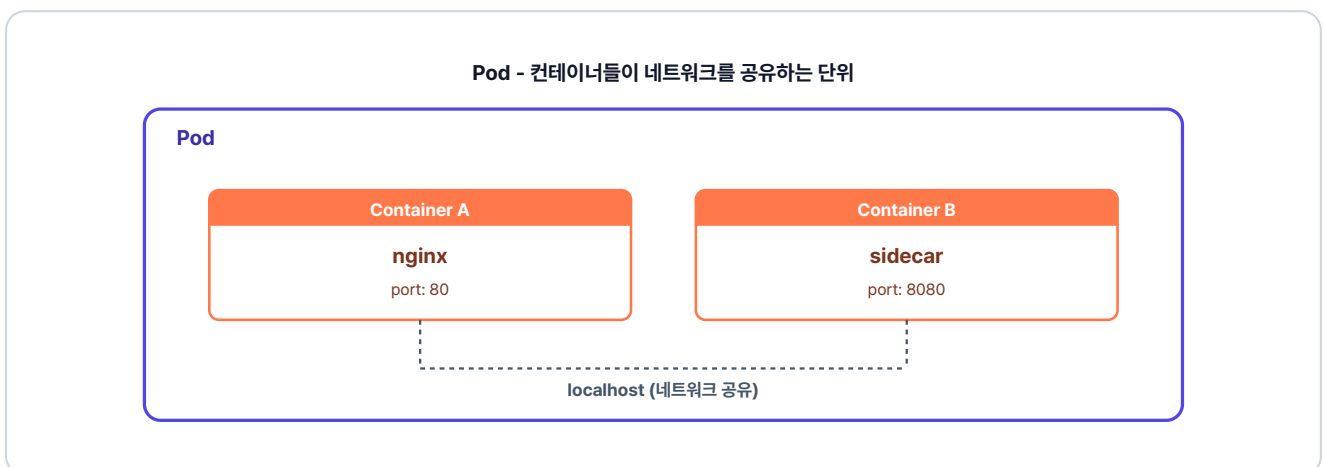


그림 4-9. Pod - 컨테이너들이 네트워크를 공유하는 단위

프랜차이즈 비유로 보면, Pod는 **가맹점 하나**와 같습니다. 가맹점 안에는 **직원(컨테이너)** 이 있고 조리 도구도 있지만, 외부에서는 **가맹점**이라는 단위로 소통하고 관리합니다.

왜 컨테이너 대신 Pod를 쓸까요

컨테이너는 서로 독립적으로 동작해 네트워크·스토리지·생명주기를 함께 관리하기 어렵습니다. 그래서 쿠버네티스는 하나처럼 묶어 관리할 수 있는 Pod를 '배포·운영의 최소 단위'로 사용합니다.

- **하나처럼 묶어서 관리:** 여러 컨테이너를 하나로 묶어 같이 생성·종료되고, 함께 동작하도록 합니다.
- **네트워크 공유:** Pod 내부 컨테이너들은 하나의 IP를 공유해 같은 서버 안처럼 통신합니다.
- **관리 기준 단위:** 스케줄링, 확장, 장애 복구 등은 컨테이너가 아니라 Pod 기준으로 이루어집니다.

이러한 이유로 쿠버네티스는 컨테이너가 아닌 Pod 단위로 관리합니다.

4.3.2 명령어로 Pod 만들기

쿠버네티스에서 컨테이너가 어떻게 돌아가는지 직접 확인해 볼 차례입니다. 가장 먼저 시도한 방식은 도커와 비슷한 명령어 기반의 생성입니다.

```
kubectl run hello-pod1 --image=nginx # nginx 이미지로 Pod 생성
```

실행 결과

```
$ kubectl run hello-pod1 --image=nginx
pod/hello-pod1 created
```

그림 4-10. kubectl run으로 Pod 생성

명령어를 입력하자 쿠버네티스의 컨트롤 플레인이 클러스터 내 노드 중 하나를 선택해 Pod를 배치했습니다.

'docker run을 쓰던 것과 비슷하네. 명령을 내리면 본사(컨트롤 플레인)가 알아서 적절한 위치를 잡아 주는구나.'

4.3.3 YAML로 Pod 만들기

전체 실습 코드는 깃헙을 참고합니다.

실습 코드 (GitHub): <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex08>

명령어 한 줄은 간편합니다. 하지만 실무에서는 설계도 역할을 하는 **YAML 파일**을 주로 사용합니다. 파일로 남겨 두면 같은 설정을 재현하거나 팀원들과 공유하기가 훨씬 수월하기 때문입니다.

ex08/hello-pod2.yml

```
apiVersion: v1
kind: Pod                # 리소스 종류
metadata:
  name: hello-pod2       # 리소스명
spec:
  containers:
    - name: hello-container
      image: nginx:1.20   # 이미지
```

이렇게 작성한 파일을 클러스터에 적용할 때는 **kubectl apply** 명령어를 사용합니다.

```
kubectl apply -f ex08/hello-pod2.yml # YAML로 Pod 생성
```

실행 결과

```
$ kubectl apply -f ex08/hello-pod2.yml
pod/hello-pod2 created
```

그림 4-11. kubectl apply로 Pod 생성

'명령어 한 줄보다는 파일로 관리하는 쪽이 나중에 다시 보기에, 팀에 공유하기에도 편하겠다.'

4.3.4 Pod 조회

생성 명령을 내렸다면 이제 상태를 확인할 차례입니다.

```
kubectl get pod # Pod 목록 조회
```



실행 결과

```
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
hello-pod1 1/1 Running 0 8m39s
hello-pod2 1/1 Running 0 23s
```

그림 4-12. Pod 목록 조회 결과

STATUS 칸에 Running이 찍혀 있습니다.

'Pod 두 개가 떴다.'

만약 Pod가 어느 노드에 배치되었는지, 혹은 생성 과정에서 어떤 일이 있었는지 상세히 알고 싶다면 describe 명령어를 사용합니다.

```
kubectl describe pod hello-pod2 # Pod 상세 정보 조회
```

실행 결과

```

$ kubectl describe pod hello-pod2
Name: hello-pod2
Namespace: default
Priority: 0
Service Account: default
Node: minikube/192.168.49.2
Start Time: Sun, 15 Mar 2026 15:41:30 +0900
Labels: <none>
Annotations: <none>
Status: Running
IP: 10.244.0.43
IPs:
  IP: 10.244.0.43
Containers:
  hello-container:
    Container ID: docker://65158d189989f41de190d076d953eb0da58c0f28cbfed088da7d91018bdc305
    Image: nginx:1.20
    Image ID: docker-
pullable://nginx@sha256:38f8c1d9613f3f42e7969c3b1dd5c3277e635d4576713e66453c6193e66270a6d
    Port: <none>
    Host Port: <none>
    State: Running
      Started: Sun, 15 Mar 2026 15:41:31 +0900
    Ready: True
    Restart Count: 0
    Environment: <none>
    Mounts:

```

그림 4-13. Pod 상세 조회

4.3.5 자주 쓰는 kubectl 명령어

오픈이는 실습 중 자주 쓰게 될 기본 명령어들을 간단히 정리해 두었습니다.

명령어	설명
<code>kubectl apply -f <파일></code>	YAML로 리소스 생성/업데이트
<code>kubectl get <리소스></code>	리소스 목록 조회
<code>kubectl describe <리소스> <이름></code>	리소스 상세 정보
<code>kubectl delete <리소스> <이름></code>	리소스 삭제
<code>kubectl exec -it <Pod명> -- bash</code>	Pod 내부 접속
<code>kubectl logs <Pod명></code>	Pod 로그 확인

노트북에 두 개의 Pod가 나란히 뒀습니다. 그런데 팀장이 던졌던 첫 번째 질문이 다시 마음에 걸렸습니다.

'근데 이것도 죽어 버리면 도커랑 차이가 없는데?'

4.4 Deployment - 자동 복구·스케일링·무중단 배포

4.4.1 Pod 하나를 직접 만들면 생기는 문제

방금 만든 Pod로 작은 실험을 해 봤습니다. 운영 중에 누군가의 실수로 Pod가 삭제되거나, 프로그램 오류로 종료된다면 어떤 일이 벌어질까요.

```
kubectl delete pod hello-pod1    # hello-pod1 삭제
kubectl get pod                  # 남은 Pod 목록 확인
```



실행 결과

```
$ kubectl delete pod hello-pod1
pod "hello-pod1" deleted
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
hello-pod2 1/1 Running 0 19m
```

그림 4-14. Pod 삭제 후 목록을 조회하면 hello-pod1이 사라져 있습니다

hello-pod1은 목록에서 사라졌고, 시간이 지나도 다시 나타나지 않았습니다.

'이게 팀장님이 말씀하신 상황이구나.'

비유로 풀어 보면, 가맹점 하나가 폐업했는데도 아무도 신경 쓰지 않는 것과 같습니다. 직접 만든 Pod는 일회성이라, 문제가 생겨 사라져도 자동으로 복구해 주는 책임자가 없습니다.

오픈이에게 필요한 건 "가맹점 개수를 항상 일정하게 관리하라" 는 명확한 지침이었습니다. 쿠버네티스에서 이 역할을 담당하는 리소스가 바로 **Deployment(디플로이먼트)** 입니다.

Deployment: Pod의 생성, 개수 유지, 업데이트를 자동으로 관리하는 리소스입니다. 실무에서는 장애 복구(Self-healing) 기능을 위해 Pod를 직접 만들지 않고 Deployment 같은 컨트롤러로 감싸서 배포합니다. selector는 관리 대상을 식별하고, template은 새로 찍어낼 Pod의 규격을 정의합니다.

4.4.2 Deployment - 본사의 지침서

전체 실습 코드는 깃헙을 참고합니다.

실습 코드 (GitHub): <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex09>

Deployment는 "Pod를 몇 개 유지할지, 문제가 생기면 어떻게 교체할지"를 적어 두는 매뉴얼과 같습니다. 본사가 운영 지침서를 내려 주면 현장 운영팀이 그 지침대로 가맹점 개수를 맞추는 구조입니다.

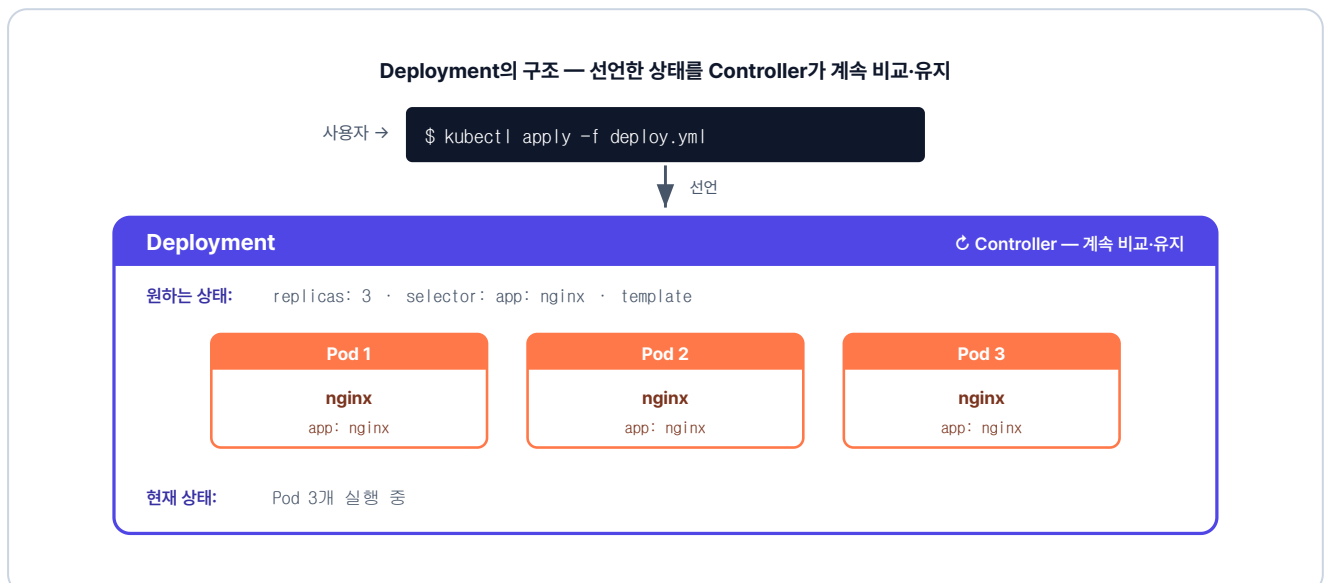


그림 4-15. Deployment의 구조 — Controller가 원하는 상태와 현재 상태를 계속 비교·유지

오픈이는 ex09 폴더에서 nginx Pod 한 개를 항상 유지하는 Deployment YAML을 확인했습니다.

ex09/deploy-ex01.yml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  # pod에 대한 상태 지정
  replicas: 1      # 생성할 pod 수 지정
  selector:
    matchLabels:
      app: nginx    # 'app: nginx' 라벨이 붙은 Pod를 내 관리 대상으로 지정
  template:
    metadata:
      labels:
        app: nginx  # pod에 붙일 라벨
    spec:
      containers:
        - name: nginx-container
          image: nginx:1.20
```

여기서 가장 중요한 개념은 **selector**와 **labels**의 연결입니다. Deployment의 selector에 **app: nginx**라고 적어 두면, 쿠버네티스는 해당 라벨을 가진 Pod들만 골라 관리합니다. 이 라벨 매칭 방식은 Deployment만 쓰는 게 아닙니다. 다음 챕터에서 만날 **Service**도 같은 라벨로 Pod를 찾습니다.

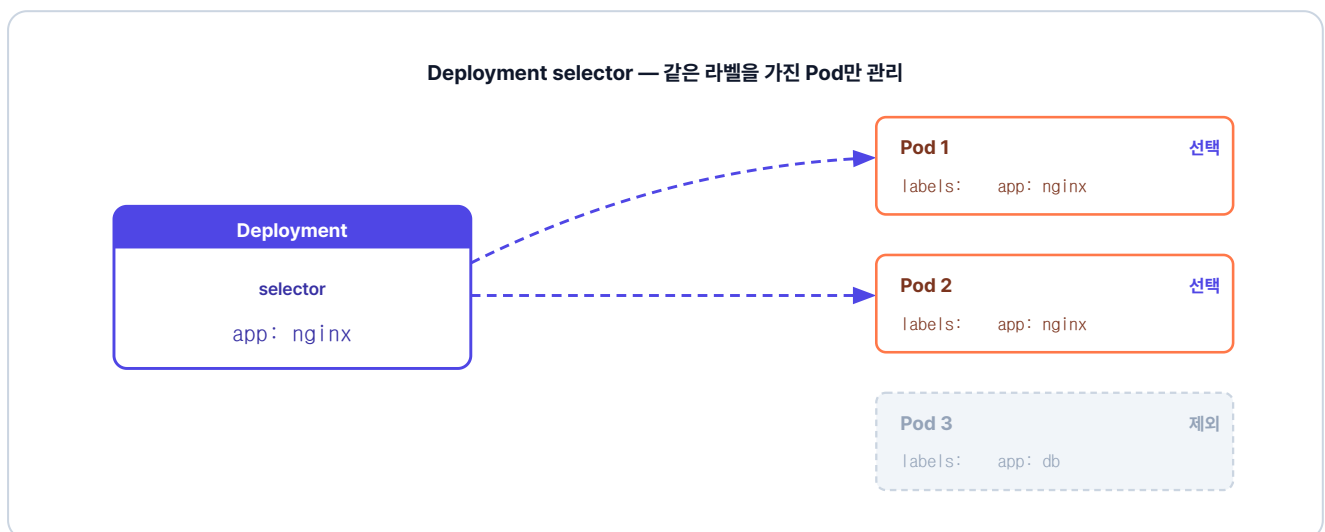


그림 4-16. selector가 지정한 라벨(app: nginx)을 가진 Pod만 골라 관리, 다른 라벨은 건드리지 않음

작성한 파일을 적용해 보겠습니다.

```
kubectl apply -f ex09/deploy-ex01.yml # Deployment 생성
kubectl get pod # Pod 목록
```



실행 결과

```
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
hello-pod2 1/1 Running 0 19m
nginx-deploy-756b46b54c-8q8p1 1/1 Running 0 5s
```

그림 4-17. Deployment가 만든 Pod가 뜬 모습

이제 Deployment가 정말로 지침을 지키는지 확인하기 위해, 현재 떠 있는 Pod를 전부 강제로 지워 봤습니다.

```
kubectl delete pod --all # 모든 Pod 삭제
kubectl get pod # 목록 재확인
```



실행 결과

```
$ kubectl delete pod --all
pod "hello-pod2" deleted
pod "nginx-deploy-756b46b54c-8q8p1" deleted
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
nginx-deploy-756b46b54c-2sv8x 1/1 Running 0 3m11s
```

그림 4-18. Pod를 다 지워도 Deployment가 자동으로 새 Pod를 띄웁니다

`hello-pod2` 는 그대로 사라졌지만 Deployment가 만든 Pod는 잠깐 사라졌다가 **새 이름으로 다시 올라와** 있었습니다.

'오, 이거다. 내가 수동으로 살릴 필요가 없네.'

Deployment에 replicas: 1이라고 선언해 두었기 때문에, Pod가 사라지면 시스템이 이를 감지하고 즉시 설계도(template)대로 새 Pod를 만들어 냅니다. 도커를 쓸 때 오픈이가 일일이 확인하며 다시 띄워야 했던 일을 이제 쿠버네티스가 대신 합니다.

4.4.3 ReplicaSet - Pod 개수 자동 관리

전체 실습 코드는 깃헙을 참고합니다.

실습 코드 (GitHub): <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex10>

'하나가 살아나는 건 알겠는데, 사용자가 동시에 몰리면 어떨까. 아무리 성능이 좋아도 한 대로는 한계가 있을 텐데.'

운영 환경에서는 부하를 분산하기 위해 여러 대의 Pod를 띄워야 합니다. 쿠버네티스에서는 replicas 값만 수정하면 되는데, 이 개수 관리를 실질적으로 담당하는 리소스가 바로 **ReplicaSet(레플리카셋)** 입니다.

에어컨이 설정 온도를 유지하기 위해 실시간으로 온도를 체크하듯, ReplicaSet은 **사용자가 선언한 개수 (Desired State)** 와 **실제 돌아가는 개수**를 계속 비교합니다. 누군가 Pod를 지우거나 장애가 생겨 숫자가 모자라면, 즉시 새 Pod를 생성해 선언한 개수를 맞춥니다.

ReplicaSet: Deployment가 내부적으로 생성·관리하는 리소스로, 실제 Pod 개수 유지의 실행 주체입니다. 사용자는 보통 ReplicaSet을 직접 만들지 않고 Deployment를 통해 관리합니다.

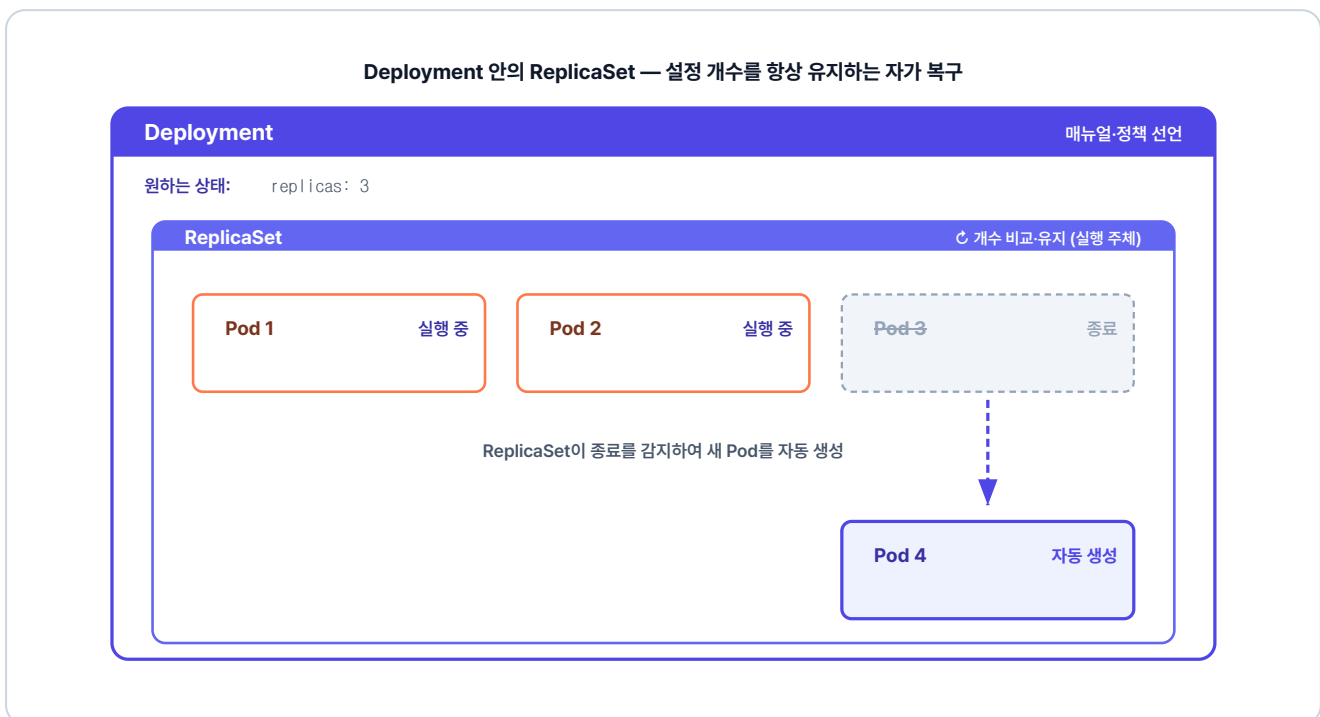


그림 4-19. Pod 하나가 종료되면 ReplicaSet이 설정 개수를 맞추기 위해 새 Pod를 자동 생성

replicas를 4로 설정해 실행될 Pod의 개수를 지정했습니다.

ex10/deploy-ex02.yml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-replica
spec:
  replicas: 4          # pod 수 지정

  strategy:
    type: RollingUpdate # 롤링 업데이트 전략
    rollingUpdate:
      maxSurge: 4      # 업데이트 중 최대 4개까지 추가 생성
      maxUnavailable: 0 # 기존 Pod를 먼저 종료하지 않음 (무중단 배포)

  selector:
    # 라벨이 app: nginx 인 pod를 관리
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx      # pod에 붙일 라벨
    spec:
      containers:
        - name: nginx-container
          image: nginx:1.20
```

이 YAML 파일을 클러스터에 적용하고, 설정한 대로 4개의 Pod가 생성되는지 확인했습니다.

```
kubectl apply -f ex10/deploy-ex02.yml # Deployment YAML 적용
kubectl get pod                        # 생성된 Pod 목록 확인
```



실행 결과

```
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
nginx-replica-756b46b54c-l5l7f 1/1 Running 0 5s
nginx-replica-756b46b54c-t8fhl 1/1 Running 0 5s
nginx-replica-756b46b54c-vp5g5 1/1 Running 0 5s
nginx-replica-756b46b54c-vzv7n 1/1 Running 0 5s
```

그림 4-20. replicas 설정대로 Pod 4개가 생성

'replicas 설정값만 바꿨는데 자리가 4개나 생기네. 서버도 쉽게 늘릴 수 있으니 부하 걱정은 한결 가벼워졌어.'

4.4.4 롤링 업데이트 - 끊임 없는 버전 교체

'그런데 4개나 되는 Pod를 새 버전으로 바꾸려면 어떻게 하지. 전부 내리고 새로 올리면 그 찰나에 서비스가 죽을 텐데. 팀장님한테 혼나겠지.'

스케일링으로 동시 접속 문제는 해결했지만, 다음 고민은 **배포**였습니다. 서버를 내리고 새 버전으로 한꺼번에 바꾸면, 서버가 내려가 있는 동안 오류가 발생합니다.

이 지점에서 등장하는 것이 **롤링 업데이트(Rolling Update)** 입니다. 새 버전을 먼저 띄우고, 제대로 작동하는 게 확인되면 기존 버전을 내리는 식으로 **순차 교체**하는 방식입니다. YAML 파일의 strategy 설정이 바로 이 교체 속도를 조절하는 리모컨입니다.

RollingUpdate 전략: K8s Deployment의 기본 배포 전략입니다. `maxSurge` 와 `maxUnavailable` 두 값으로 "몇 개를 더 띄울 수 있는지", "몇 개까지 사용 불가능해도 되는지"를 조정합니다.

위 YAML의 `strategy` 블록의 속성은 다음과 같습니다.

- **maxSurge:** 업데이트 중 정원(replicas)보다 잠시 증가되는 Pod의 수 (추가 투입 인원)
- **maxUnavailable:** 업데이트 중 **작동 불능** 상태가 되어도 허용되는 Pod의 수 (잠시 문 닫아도 되는 가게 수)

예를 들어 replicas: 4, maxSurge: 4, maxUnavailable: 0이라면, 기존 4개는 그대로 둔 상태에서 새 버전 4개를 한꺼번에 더 만듭니다. 순식간에 가맹점이 8개로 늘어나는 구조입니다. 새 Pod가 손님을 받을 준비(Ready) 상태가 되는 대로 기존 Pod를 차례로 종료시킵니다. maxUnavailable: 0 덕분에 어떤 순간에도 최소 4대가 살아 있습니다. 그래서 손님은 배포 중인지도 모르는 **무중단 배포**가 됩니다.

명령어 한 줄로 이미지 버전을 올려 봤습니다.

```
kubectl set image deployment/nginx-replica nginx-container=nginx:1.21 # 이미지 1.21로 교체
```



그림 4-21. 이미지 버전 업데이트 실행

실시간으로 지켜보면 새 Pod 4개가 먼저 모두 떠오른 뒤, 기존 Pod 4개가 함께 종료되는 순서를 확인할 수 있습니다. ex10의 strategy가 `maxSurge: 4` 와 `maxUnavailable: 0` 이라, 새 Pod가 모두 준비된 다음 기존 Pod가 내려갑니다.

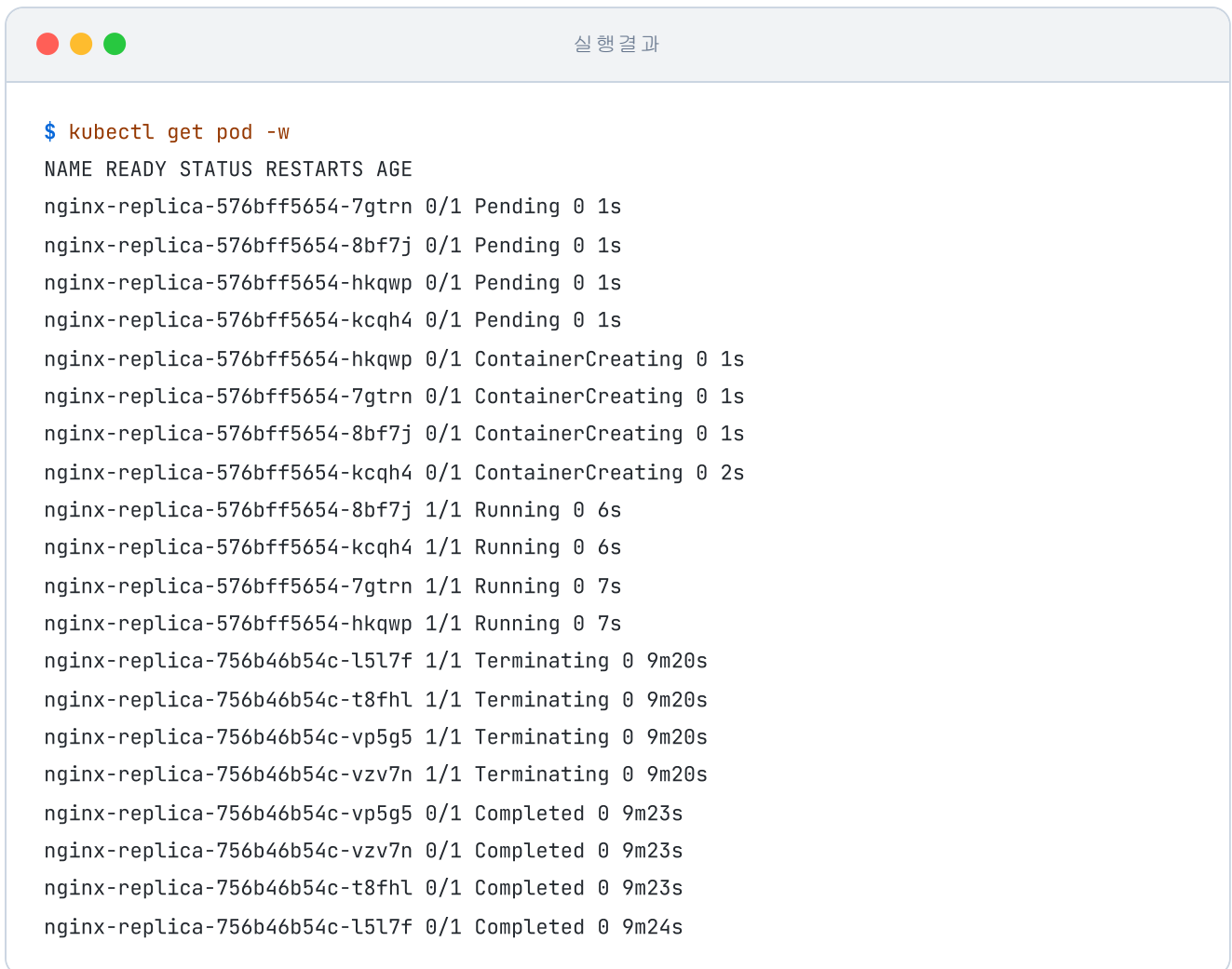
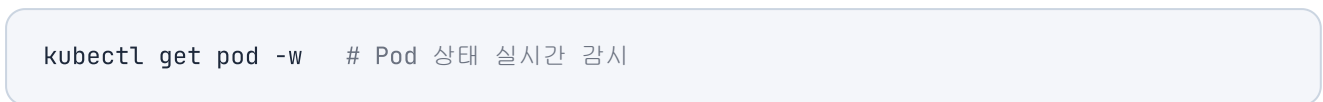


그림 4-22. 롤링 업데이트 진행 화면

출력에서 새 Pod는 **Pending → ContainerCreating → Running** 순서로 실행되고, 기존 Pod는 **Running → Terminating → Completed** 순서로 종료됩니다.

업데이트가 끝난 뒤 Deployment의 상세 정보를 확인하면 이미지가 `nginx:1.21` 버전으로 변경된 것을 볼 수 있습니다.

```
kubectl describe deployment nginx-replica # Deployment 상세 정보 조회
```

4.4.5 Rollback - 되돌리기

세상에 완벽한 배포는 없습니다. 만약 새 버전이 문제가 보인다면, 쿠버네티스는 직전 버전으로 되돌리는 명령을 마련해 두었습니다.

```
kubectl rollout history deployment/nginx-replica # 배포 이력 조회
kubectl rollout undo deployment/nginx-replica # 이전 버전으로 롤백
```

실행 결과

```
$ kubectl rollout undo deployment/nginx-replica
deployment.apps/nginx-replica rolled back
```

그림 4-23. Rollback 실행 결과

배포가 꼬여도 단 두 줄이면 직전 버전으로 되돌릴 수 있습니다.

'휴, 배포 실패 복구가 이렇게 간단하다니. 이제 배포 날 새벽까지 떨지 않아도 되겠어.'

이것만은 기억하자

- **Docker**는 "지금 띄워라", **Kubernetes**는 "이 상태를 유지해라". 선언형 관리의 핵심입니다. 내가 원하는 상태를 YAML에 정의해 두면, 쿠버네티스가 실시간 상태를 확인하며 그 상태에 계속 맞춥니다.
- **K8s**는 **프랜차이즈 본사 구조**입니다. 본사가 전국의 가맹점을 관리하는 모습과 같습니다. 컨트롤 플레인(관리자)이 워커 노드(가맹점) 위의 Pod를 관리합니다.
- **Pod**는 가맹점, 실행의 최소 단위입니다. K8s에서 컨테이너는 Pod라는 상자에 담겨 움직입니다.

- **Pod를 직접 만들지 마세요.** 직접 만든 Pod는 장애가 나도 복구되지 않습니다. 반드시 Deployment로 감싸서 관리 목록에 올려야 자동 복구와 스케일링이 가능해집니다.
- **롤링 업데이트로 무중단 배포를 합니다.** 새 Pod를 먼저 띄운 뒤 기존 Pod를 순차적으로 교체하여, 서비스 중단 없이 버전을 올리는 방식입니다.

Pod와 Deployment로 자동 복구·스케일링·무중단 배포까지 손에 넣었습니다. 다만 외부에서 이 Pod에 들어올 방법은 아직 없습니다. 다음 챕터에서는 Service와 Ingress로 외부와 연결되는 진입점을 만들어 보겠습니다.